Slip2Go API Documentation

REST API · Queue API · Authentication · Response Codes

Source	https://slip2go.com/guide (all 11 pages in the guide's left-hand menu)
Prepared	27 October 2026, from the live website
Nature of document	Unofficial English rendering of the Thai-language Slip2Go guide
Left out on purpose	JavaScript and curl usage examples (not requested)

> **Please note.** The Slip2Go website remains the authoritative source. Section 7 lists the few places where the guide itself is silent or ambiguous; check those before going live.

Contents

1. Overview
2. Authentication
3. Response Codes
4. Shared Definitions (conditions, result object, account types)
5. REST API
   - 5.1 Verify Slip by QR Code
   - 5.2 Verify Slip by Image
   - 5.3 Verify Slip by Base64
   - 5.4 Verify Slip by Image URL
   - 5.5 Generate PromptPay QR Code
   - 5.6 Generate PromptPay QR Image URL
   - 5.7 Retrieve a Previous Slip
   - 5.8 Retrieve Account Info
6. Queue API
   - 6.1 Verify Slip by QR Code
7. Notes on Source Coverage

1. Overview

Slip2Go's API Connect lets a system verify Thai bank transfer slips and generate PromptPay QR codes. A slip can be submitted as a QR code string, an image file, a Base64 image, or an image URL. Optional conditions (duplicate slip, receiver account, amount, date) can be checked in the same call.

The documentation covers two services. The **REST API** (Section 5) is synchronous and groups eight endpoints. The **Queue API** (Section 6) is a separate, asynchronous service that answers by callback. Authentication (Section 2), result codes (Section 3) and shared definitions (Section 4) apply to both.

1.1 Endpoint summary

#	Purpose	Method	Path	Service / Section
1	Verify slip by QR code	POST	`/api/verify-slip/qr-code/info`	REST API 5.1
2	Verify slip by image file	POST	`/api/verify-slip/qr-image/info`	REST API 5.2
3	Verify slip by Base64 image	POST	`/api/verify-slip/qr-base64/info`	REST API 5.3
4	Verify slip by image URL	POST	`/api/verify-slip/qr-image-link/info`	REST API 5.4
5	Generate PromptPay QR code string	POST	`/api/qr-payment/generate-qr-code`	REST API 5.5
6	Generate PromptPay QR image URL	POST	`/api/qr-payment/generate-qr-image-link`	REST API 5.6
7	Retrieve a previously verified slip	GET	`/api/verify-slip/{referenceId}`	REST API 5.7
8	Retrieve account / package info	GET	`/api/account/info`	REST API 5.8
9	Queue API: verify slip by QR code	POST	`/api/queue/verify-slip/qr-code`	Queue API 6.1

> **Base URL.** The guide lists paths only and does not state a base URL. Obtain it from your Slip2Go account or from Slip2Go support.

1.2 Conventions used by every endpoint

- Every endpoint requires authentication: send your API Secret in the `Authorization` header (Section 2).
- An IP whitelist can optionally restrict which IP addresses may call the API.
- Request bodies are JSON, except the image-upload endpoint (Section 5.2), which uses `multipart/form-data`.
- Responses use a common envelope: `code`, `message` and (when applicable) `data`.
- Slip-verification outcomes such as 'slip not found' or 'recipient account does not match' are returned with **HTTP 200** and a six-digit `code` starting with `2004` or `2005`. Always branch on `code`, not on the HTTP status alone.
- Dates are ISO 8601 strings in GMT, e.g. `2025-10-05T14:48:00.000Z`.

2. Authentication

Authentication identifies the caller of the API. All endpoints in this document are marked 'Auth Required'.

2.1 Using your Secret

- Add a header named `Authorization` to every request.
- Set its value to your {Slip2Go_Secret_Key.}

2.2 IP whitelist

- An IP whitelist limits API access to the IP addresses you list.
- Up to **10** IP addresses can be set.
- The default value is `*`, which accepts every IP address. You can remove it.
- A call from an address that is not allowed fails with code `401007` (IP Address not Allowed).

2.3 Related error codes

Code	HTTP	Meaning
`401001`	401	Authentication Token Mismatch: the token is invalid.
`401007`	401	IP Address not Allowed.
`401002` to `401006`	401	Shop not found, account suspended, package expired, out of tokens, or insufficient credit (full list in Section 3).

> **Verify before use.** The guide text says only that the header value equals the Secret Key; its JavaScript and curl samples are collapsed on the page and are not reproduced here. A third-party integration report states that the value must be sent as `Bearer <secret>` and that the raw secret returned `401001`. This is not confirmed by the guide text, so check the curl sample on https://slip2go.com/guide/authentication for the exact header format.

3. Response Codes

This section lists the result codes returned in the `code` field. The generate-QR endpoints (Sections 5.5 and 5.6) return their own simple `"200"` / `"Success"` result.

3.1 Success responses (HTTP 200)

Code	Status (message)	Meaning	Token charged
200000	Slip found	The slip data exists in the bank system and is correct.	Yes
200001	Get Info Success	Information request succeeded.	No
200200	Slip is valid	The slip data is valid.	Yes
200202	Successfully Queue	Data was saved to the queue (Queue API).	Yes
200401	Recipient Account Not Match	The receiver account does not match the condition.	Yes
200402	Transfer Amount Not Match	The transferred amount does not meet the condition.	Yes
200403	Transfer Date Not Match	The transfer date does not meet the condition.	Yes
200404	Slip not found	The slip was not found in the bank system.	Yes
200500	Slip is fraud	The slip is invalid or forged.	Yes
200501	Slip is Duplicated	The slip was already submitted.	No
200502	Bank error, Please try again	The error came from the bank.	No

> **About 'Token charged'.** The original page shows this column as an icon only. The Yes/No values above are read from that icon; confirm against Slip2Go's pricing page before relying on them for billing.

3.2 Error responses

Code	HTTP	Status (message)	Meaning
400001	400	QR Code is missing	The QR code is invalid.
400002	400	File is Incorrect	The file is invalid.
400003	400	Reference Id Not found	The reference ID was not found.
400004	400	Image url format is not Valid	The image URL format is invalid.
400005	400	Base64 format is not Valid	The Base64 format is invalid.
400400	400	Request object is invalid	The verification (condition) data is invalid.
400409	400	Request is conflicted	A duplicate or conflicting request was sent.
400422	400	Webhook URL is required for this service	The Queue API requires a Webhook URL to be set first.

Code	HTTP	Status (message)	Meaning
401001	401	Authentication Token Mismatch	The token is invalid.
401002	401	Shop not found	The shop or branch was not found.
401003	401	Account not found	The account is suspended.
401004	401	Package Expired	The package has expired.
401005	401	Insufficient Token	Slip-verification tokens have run out.
401006	401	Insufficient Credit	Credit is not sufficient.
401007	401	IP Address not Allowed	The calling IP address is not allowed.
429000	429	Too many requests, API rate limit exceeded	The number of requests exceeded the system limit.
500500	500	Internal server error	A problem in the bank system or the Slip2Go system.
500503	503	The system is undergoing scheduled maintenance.	The system is under scheduled maintenance.

4. Shared Definitions

The verification endpoints share the same conditions object, the same result object and the same list of account types. They are defined once here and referenced from the endpoint sections.

4.1 Slip conditions (checkCondition)

Where it goes: `payload.checkCondition` in Sections 5.1, 5.3, 5.4 and 6.1. In Section 5.2 the same keys are sent inside the `payload` form field. All keys are optional; send only the checks you need.

Field	Type	Required	Description
`checkDuplicate`	Boolean	No	Set to `true` to check whether the slip was already submitted.
`checkReceiver`	Array	No	Receiver-account checks. Pass several objects to check several accounts at once; the slip is accepted if at least one object matches.
`checkReceiver[].accountType`	String	No	Account type code to check. See Section 4.3.
`checkReceiver[].accountNameTH`	String	No	Receiver name in Thai script. Do not include a title or prefix.
`checkReceiver[].accountNameEN`	String	No	Receiver name in English. Do not include a title or prefix. Example: `"Somchay Slip2go"`.
`checkReceiver[].accountNumber`	String	No	Account number to check. No spaces or symbols. Example: `"xxxxxx1234"`.
`checkAmount`	Object	No	Transfer-amount check.
`checkAmount.type`	String	No	Comparison: `"lte"` (less than or equal), `"eq"` (equal), `"gte"` (greater than or equal). Defaults to `"eq"` when omitted.
`checkAmount.amount`	String	No	Amount to compare against, as a plain string such as `"10000"`, without commas. (The guide also says not to include '0'; this most likely means no padding or decimal zeros.)
`checkDate`	Object	No	Transfer-date check.
`checkDate.type`	String	No	Comparison: `"lte"`, `"eq"` or `"gte"`. Defaults to `"eq"` when omitted.
`checkDate.date`	DateISO	No	Transfer date in ISO 8601, GMT. Example: `"2025-10-05T14:48:00.000Z"`.

Example of the object (several checks combined):

```
{
  "checkDuplicate": true,
  "checkReceiver": [
    { "accountType": "01004", "accountNumber": "xxxxxx1234" }
  ],
  "checkAmount": { "type": "gte", "amount": "100" },
```

```
  "checkDate": { "type": "eq", "date": "2025-10-05T14:48:00.000Z" }
}
```

4.2 Verification result object

Returned by Sections 5.1 to 5.4. Section 5.7 returns the same object plus `verifyDate`; Section 6.1 delivers it by callback with `queueId` in place of `referenceId`.

Field	Type	Description
`code`	String	Result code of the operation (see Section 3), e.g. `"200000"`.
`message`	String	Result message, e.g. `"Slip found"`.
`data`	Object | undefined	Result data. Contents are described below.
`data.referenceId`	String	Slip reference ID. Use it to retrieve the slip later (Section 5.7).
`data.decode`	String	Reference string of the slip's QR code.
`data.transRef`	String	Reference code of this transaction record.
`data.dateTime`	String	Transfer date and time (ISO 8601, GMT).
`data.amount`	Number	Amount transferred.
`data.ref1` / `ref2` / `ref3`	String | null	Reference codes 1, 2 and 3 (null when absent).
`data.receiver`	Object	Receiver information.
`data.receiver.account`	Object	Receiver account information.
`data.receiver.account.name`	String	Receiver account name.
`data.receiver.account.bank`	Object	Receiver bank-account information.
`data.receiver.account.bank.account`	String | null	Receiver bank account number (masked, e.g. `xxx-x-x5366-x`).
`data.receiver.account.proxy`	Object | null	Receiver proxy (PromptPay ID) information, if the transfer used one.
`data.receiver.account.proxy.type`	String | null	Proxy type: `NATID`, `MSISDN`, `EWALLTID`, `EMAIL` or `BILLERID`.
`data.receiver.account.proxy.account`	String | null	Receiver proxy number (masked).
`data.receiver.bank`	Object	Receiver bank information.
`data.receiver.bank.id`	String	Receiver bank code, e.g. `"004"`.
`data.receiver.bank.name`	String | null	Receiver bank name.
`data.sender`	Object	Sender information.
`data.sender.account`	Object	Sender account information.
`data.sender.account.name`	String	Sender name.
`data.sender.account.bank`	Object	Sender bank-account information.
`data.sender.account.bank.account`	String	Sender bank account number (masked).

Field	Type	Description
`data.sender.bank`	Object	Sender bank information.
`data.sender.bank.id`	String	Sender bank code.
`data.sender.bank.name`	String | null	Sender bank name.

Sample response


```
{
  "code": "200000",
  "message": "Slip found.",
  "data": {
    "referenceId": "92887bd5-60d3-4744-9a98-b8574eaxxxxx-xx",
    "decode": "00410006000001010300402200142420825 47BPM049885102TH9104xxxx",
    "transRef": "015073144041ATF00999",
    "dateTime": "2025-10-05T14:48:00.000Z",
    "amount": 1,
    "ref1": null,
    "ref2": null,
    "ref3": null,
    "receiver": {
      "account": {
        "name": "Slip2Go Co., Ltd.",
        "bank": { "account": "xxx-x-x5366-x" },
        "proxy": { "type": "NATID", "account": "xxx-x-x5366-x" }
      },
      "bank": { "id": "004", "name": "Kasikorn Bank" }
    },
    "sender": {
      "account": {
        "name": "Slip2Go Co., Ltd.",
        "bank": { "account": "xxx-x-x9866-x" }
      },
      "bank": { "id": "004", "name": "Kasikorn Bank" }
    }
  }
}
```


4.3 Account type codes

Used in `checkReceiver[].accountType`.

Code	Bank or account type
`01002`	Bangkok Bank
`01004`	Kasikorn Bank
`01006`	Krung Thai Bank
`01008`	JPMorgan Chase Bank (JPMC)
`01011`	TMB Thanachart Bank
`01014`	Siam Commercial Bank (SCB)
`01017`	Citibank (CITI)
`01018`	Sumitomo Mitsui Bank (SMBC)
`01020`	Standard Chartered Bank (Thai) (SCBT)
`01022`	CIMB Thai Bank
`01024`	United Overseas Bank (UOB)
`01025`	Bank of Ayudhya (Krungsri)

Code	Bank or account type
01029	Indian Overseas Bank (IOBA)
01030	Government Savings Bank (GSB)
01031	HSBC
01032	Deutsche Bank AG (DB)
01033	Government Housing Bank (GHB)
01034	Bank for Agriculture and Agricultural Cooperatives (BAAC)
01039	Mizuho Bank (MIZUHO)
01045	BNP Paribas (BNPP)
01052	Bank of China (Thai) (BOC)
01066	Islamic Bank of Thailand
01067	TISCO Bank
01069	Kiatnakin Phatra Bank
01070	Industrial and Commercial Bank of China (Thai) (ICBC Thai)
01071	Thai Credit Bank
01073	Land and Houses Bank (LH Bank)
01088	CLICX Bank
01098	Small and Medium Enterprise Development Bank (SME Bank)
02001	PromptPay: mobile phone number
02003	PromptPay: national ID / tax ID
02004	PromptPay: e-Wallet ID
03000	Merchant accounts: K+ Shop (KBANK), Mae Manee (SCB), Be Merchant NextGen (BBL), TTB Smart Shop (TTB), Nong Hom Jang shop (BAAC)
04000	True Money Wallet

Source difference. Code `01002` (Bangkok Bank) appears in the lists on the Image, Base64, Image URL and Queue API pages. It was not shown in the list on the QR Code page when read. Treat it as supported but confirm if you depend on it.

5. REST API

The REST API is the synchronous Slip2Go service: each call returns its result in the response. It groups eight endpoints, described in the subsections below. All of them use the authentication in Section 2, the result codes in Section 3 and the shared definitions in Section 4.

Endpoints in this group

Section	Purpose	Method	Path
5.1	Verify slip by QR code	POST	`/api/verify-slip/qr-code/info`
5.2	Verify slip by image file	POST	`/api/verify-slip/qr-image/info`
5.3	Verify slip by Base64 image	POST	`/api/verify-slip/qr-base64/info`
5.4	Verify slip by image URL	POST	`/api/verify-slip/qr-image-link/info`
5.5	Generate PromptPay QR code string	POST	`/api/qr-payment/generate-qr-code`
5.6	Generate PromptPay QR image URL	POST	`/api/qr-payment/generate-qr-image-link`
5.7	Retrieve a previously verified slip	GET	`/api/verify-slip/{referenceId}`
5.8	Retrieve account / package info	GET	`/api/account/info`

5.1 Verify Slip by QR Code

Verifies a transfer slip from the QR code string printed on the slip.

Endpoint	`POST /api/verify-slip/qr-code/info`
Authentication	Required (Section 2)
Content type	JSON request body

5.1.1 Request body

Field	Type	Required	Description
`payload`	Object	Yes	Container for all request data.
`payload.qrCode`	String	Yes	The QR code string from the slip, e.g. `00410006000001010300402200142420825 47BPM049885102TH9104xxxx`.
`payload.checkCondition`	Object	No	Extra slip checks. Include only when needed. See Section 4.1.

> **Slip conditions.** To check conditions, send only the keys you want checked. The fields are defined once in Section 4.1.

Example (QR code only, no conditions):

```
{
  "payload": {
    "qrCode": "00410006000001010300402200142420825 47BPM049885102TH9104xxxx"
  }
}
```

Example (QR code with conditions):

```
{
  "payload": {
    "qrCode": "00410006000001010300402200142420825 47BPM049885102TH9104xxxx",
    "checkCondition": {
      "checkDuplicate": true,
      "checkReceiver": [{ "accountType": "01004", "accountNumber": "xxxxxx1234" }],
      "checkAmount": { "type": "lte", "amount": "10000" }
    }
  }
}
```

5.1.2 Response

The response is the verification result object in Section 4.2. Result codes are listed in Section 3.

5.1.3 Usage conditions

- Send the API Secret in the header on every call.
- An IP whitelist can restrict access.
- To check slip conditions, send only the keys you want checked.

5.2 Verify Slip by Image

Verifies a transfer slip by uploading the slip image.

Endpoint	`POST /api/verify-slip/qr-image/info`
Authentication	Required (Section 2)
Content type	`multipart/form-data`

5.2.1 Request data (multipart/form-data)

Field	Type	Required	Description
`file`	File	Yes	The slip image. Accepted examples: `slip.png`, `slip.jpg`, `slip.jpeg`.
`payload`	JSON	No	Send when you want slip conditions checked. A JSON object holding the condition keys below.

In this endpoint the guide lists the condition keys at the top level of the `payload` JSON (for example `checkDuplicate` and `checkReceiver`), without the `payload.checkCondition` prefix used by the JSON endpoints. The keys, types and rules are those in Section 4.1.

> **Slip conditions.** To check conditions, send only the keys you want checked. The fields are defined once in Section 4.1.

5.2.2 Response

The response is the verification result object in Section 4.2. Result codes are listed in Section 3.

5.2.3 Usage conditions

- Send the API Secret in the header on every call.
- An IP whitelist can restrict access.
- To check slip conditions, send only the keys you want checked.

5.3 Verify Slip by Base64

Verifies a transfer slip from a Base64-encoded image.

Endpoint	`POST /api/verify-slip/qr-base64/info`
Authentication	Required (Section 2)
Content type	JSON request body

5.3.1 Request body

Field	Type	Required	Description
`payload`	Object	Yes	Container for all request data.
`payload.imageBase64`	String	Yes	The Base64 image string. The guide's example includes the data-URI prefix, e.g. `....`
`payload.checkCondition`	Object	No	Extra slip checks. See Section 4.1.

> **Slip conditions.** To check conditions, send only the keys you want checked. The fields are defined once in Section 4.1.

5.3.2 Response

The response is the verification result object in Section 4.2. Result codes are listed in Section 3. An invalid Base64 string returns `400005`.

5.3.3 Usage conditions

- Send the API Secret in the header on every call.
- An IP whitelist can restrict access.
- To check slip conditions, send only the keys you want checked.

5.4 Verify Slip by Image URL

Verifies a transfer slip from an image that is reachable at a URL.

Endpoint	`POST /api/verify-slip/qr-image-link/info`
Authentication	Required (Section 2)
Content type	JSON request body

5.4.1 Request body

Field	Type	Required	Description
`payload`	Object	Yes	Container for all request data.
`payload.imageUrl`	String	Yes	URL of the slip image, e.g. `https://xxxxxxxxx.com/slip2go-check.jpg`.
`payload.checkCondition`	Object	No	Extra slip checks. See Section 4.1.

> **Slip conditions.** To check conditions, send only the keys you want checked. The fields are defined once in Section 4.1.

5.4.2 Response

The response is the verification result object in Section 4.2. Result codes are listed in Section 3. An invalid URL returns `400004`.

5.4.3 Usage conditions

- Send the API Secret in the header on every call.
- An IP whitelist can restrict access.
- To check slip conditions, send only the keys you want checked.

5.5 Generate PromptPay QR Code

Generates a PromptPay QR code string for a given account and amount. Render the returned string as a QR image in your own application.

Endpoint	`POST /api/qr-payment/generate-qr-code`
Authentication	Required (Section 2)
Content type	JSON request body

5.5.1 Request body

Field	Type	Required	Description
`promptPayCode`	String	Yes	The PromptPay account the QR code pays into, e.g. `"0902369994"`.
`promptPayType`	String	Yes	PromptPay account type: `"phone_number"` = mobile phone number `"citizen_id"` = national ID card `"e_wallet"` = e-Wallet ID
`accountName`	String	Yes	Receiver account name.
`amount`	String	Yes	Amount to encode in the QR code, e.g. `"88.88"`.

5.5.2 Response

Field	Type	Description
`code`	String	Result code, e.g. `"200"`.
`message`	String	Result message, e.g. `"Success"`.
`data`	Object | undefined	Result data.
`data.qrCode`	String	The QR code string.
`data.accountName`	String	Receiver account name.
`data.amount`	String	Amount encoded in the QR code.

Sample response:

```
{
  "code": "200",
  "message": "Success",
  "data": {
    "qrCode": "00410006000001010300402200142420082547BPM049885102TH9104xxxx",
    "accountName": "Slip2Go Co., Ltd.",
    "amount": "88.88"
  }
}
```

> **Quota.** Account info (Section 5.8) reports a separate QR-generation quota: `quotaQrLimit` and `quotaQrRemaining`.

5.6 Generate PromptPay QR Image URL

Generates a ready-to-use image URL of a PromptPay QR code for a given account and amount.

Endpoint	`POST /api/qr-payment/generate-qr-image-link`
Authentication	Required (Section 2)
Content type	JSON request body

5.6.1 Request body

Identical to Section 5.5.1.

Field	Type	Required	Description
`promptPayCode`	String	Yes	The PromptPay account the QR code pays into, e.g. `"0902369994"`.
`promptPayType`	String	Yes	PromptPay account type: `"phone_number"` = mobile phone number `"citizen_id"` = national ID card `"e_wallet"` = e-Wallet ID
`accountName`	String	Yes	Receiver account name.
`amount`	String	Yes	Amount to encode in the QR code, e.g. `"88.88"`.

5.6.2 Response

Field	Type	Description
`code`	String	Result code, e.g. `"200"`.
`message`	String	Result message, e.g. `"Success"`.
`data`	Object | undefined	Result data.
`data.qrImageLink`	String	URL of the QR code image.
`data.accountName`	String	Receiver account name.
`data.amount`	String	Amount encoded in the QR code.

Sample response:

```
{
  "code": "200",
  "message": "Success",
  "data": {
    "qrImageLink": "https://xxxxxxxxxx.com/slip_qr_code.jpg",
    "accountName": "Slip2Go Co., Ltd.",
    "amount": "88.88"
  }
}
```

5.7 Retrieve a Previous Slip

Retrieves the data of a slip that was verified earlier, using its reference ID.

Endpoint	`GET /api/verify-slip/{referenceId}`
Authentication	Required (Section 2)
Request body	None

5.7.1 Request data (path parameter)

Field	Type	Required	Description
`referenceId`	String	Yes	The slip reference ID (`data.referenceId` from an earlier verification), e.g. `92887bd5-60d3-4744-9a98-b8574eaxxxxx-xx`.

5.7.2 Response

Same structure as the verification result object in Section 4.2, with these differences: `code` is `"200001"` and `message` is `"Get Info Success"`, and `data` has one extra field.

Field	Type	Description
`data.verifyDate`	String	Time at which the slip was submitted for verification (ISO 8601, GMT).

An unknown reference ID returns `400003` (Reference Id Not found).

> **Sample.** Same shape as Section 4.2, with `"code": "200001"`, `"message": "Get Info Success"` and the extra `verifyDate` field.

5.8 Retrieve Account Info

Returns information about the shop, the current package, and the remaining tokens, credit and QR-generation quota.

Endpoint	`GET /api/account/info`
Authentication	Required (Section 2)
Request body	None

5.8.1 Response

Field	Type	Description
`code`	String	Result code, e.g. `"200001"`.
`message`	String	Result message, e.g. `"Get Info Success"`.
`data`	Object	Shop and account information (fields below).
`data.shopName`	String	Shop name.
`data.package`	String	Current package name, e.g. `"BASIC-1"`.
`data.packageExpiredDate`	DateString	Package expiry date (ISO 8601).

Field	Type	Description
`data.tokenLimit`	Number	Total tokens in the current package.
`data.tokenRemaining`	Number	Tokens remaining.
`data.creditRemaining`	Number	Credit remaining.
`data.autoRenewalPackage`	Boolean	Whether automatic package renewal is on.
`data.checkSlipByCredit`	Boolean	Whether slips beyond the token allowance are verified using credit.
`data.quotaQrLimit`	Number	Total QR-code generation quota in the current package.
`data.quotaQrRemaining`	Number	QR-code generation quota remaining.
`data.tokenPerSlip`	Number	Tokens used per slip.
`data.estimatedQuotaSlip`	Number	Estimated number of slips that can still be verified.

Sample response:


```
{
  "code": "200001",
  "message": "Get Info Success",
  "data": {
    "shopName": "My Shop",
    "package": "BASIC-1",
    "packageExpiredDate": "2025-10-07T23:59:59.999Z",
    "tokenLimit": 400,
    "tokenRemaining": 200,
    "creditRemaining": 0,
    "autoRenewalPackage": false,
    "checkSlipByCredit": true,
    "quotaQrLimit": 100,
    "quotaQrRemaining": 10,
    "tokenPerSlip": 0.250,
    "estimatedQuotaSlip": 300
  }
}
```

6. Queue API

The Queue API is a separate, asynchronous service. A call is accepted into a queue and answered immediately with a queue ID; the verification result is delivered later to your Webhook URL. It shares authentication (Section 2), result codes (Section 3) and the shared definitions (Section 4) with the REST API, but is treated here as its own service.

Endpoints in this group

Section	Purpose	Method	Path
6.1	Queue API: verify slip by QR code	POST	`/api/queue/verify-slip/qr-code`

6.1 Verify Slip by QR Code

Submits a slip for verification through a queue. The call returns immediately with a queue ID, and the verification result is delivered later by callback to your Webhook URL.

Endpoint	`POST /api/queue/verify-slip/qr-code`
Authentication	Required (Section 2)
Content type	JSON request body
Prerequisite	A Webhook URL must be configured for the service. Otherwise the call fails with `400422`.

6.1.1 Request body

Identical to Section 5.1.1 (`payload.qrCode` is required; `payload.checkCondition` is optional, see Section 4.1).

Field	Type	Required	Description
`payload`	Object	Yes	Container for all request data.
`payload.qrCode`	String	Yes	The QR code string from the slip.
`payload.checkCondition`	Object	No	Extra slip checks. See Section 4.1.

6.1.2 Immediate response

A successful call returns `200202` with the queue ID:

```
{
  "code": "200202",
  "message": "successfully queue.",
  "data": {
    "queueId": "92887bd5-60d3-4744-9a98-b8574eaxxxxx-xx"
  }
}
```

Codes this endpoint can return directly:

Code	HTTP	Status (message)	Meaning
`200202`	200	Successfully Queue	Data was saved to the queue.
`400001`	400	QR Code is missing	The QR code is invalid.
`400002`	400	File is incorrect	The file is invalid.
`400003`	400	Reference Id is invalid	The reference ID was not found.

Code	HTTP	Status (message)	Meaning
400004	400	Image url format is not Valid	The image URL format is invalid.
400005	400	Base64 format is not Valid	The Base64 format is invalid.
400400	400	Request object is invalid	The verification data is invalid.
400409	400	Request is conflicted	A duplicate or conflicting request was sent.
400422	400	Webhook URL is required for this service	A Webhook URL must be set before using the Queue API.
401001 to 401007	401	Authentication, shop, account, package, token, credit and IP errors	Same meanings as in Section 3.2.

6.1.3 Callback

When the queued slip has been processed, the result is sent to your Webhook URL. The callback has the same structure as the verification result object (Section 4.2), with one difference: `data.referenceId` is replaced by `data.queueId` (the queue ID returned in Section 6.1.2).

Field	Type	Description
code	String	Result code, e.g. "200000" (see Section 3).
message	String	Result message, e.g. "Slip found".
data	Object | undefined	Result data.
data.queueId	String	Queue ID of the slip.
data.decode ... data.sender.bank.name	as Section 4.2	All other fields are identical to Section 4.2.

Sample callback:

```
{
  "code": "200000",
  "message": "Slip found.",
  "data": {
    "queueId": "92887bd5-60d3-4744-9a98-b8574eaxxxxx-xx",
    "decode": "00410006000001010300402200142420825 47BPM049885102TH9104xxxx",
    "transRef": "015073144041ATF00999",
    "dateTime": "2025-10-05T14:48:00.000Z",
    "amount": 1,
    "ref1": null,
    "ref2": null,
    "ref3": null,
    "receiver": {
      "account": {
        "name": "Slip2Go Co., Ltd.",
        "bank": { "account": "xxx-x-x5366-x" },
        "proxy": { "type": "NATID", "account": "xxx-x-x5366-x" }
      },
      "bank": { "id": "004", "name": "Kasikorn Bank" }
    },
    "sender": {
      "account": {
        "name": "Slip2Go Co., Ltd.",
        "bank": { "account": "xxx-x-x9866-x" }
      },
      "bank": { "id": "004", "name": "Kasikorn Bank" }
    }
  }
}
```

Not covered by the guide. The guide does not describe how callbacks are authenticated, retried, or what Webhook URL format is expected, nor where the Webhook URL is configured.

7. Notes on Source Coverage

These points come from comparing the written guide with what is stated on its pages. None changes the field definitions above, but each is worth a check before production use.

- **Code samples.** The JavaScript and curl samples and the 17 named request-body examples on each verification page are collapsed behind buttons on the website and are not included here (usage examples were out of scope). The request examples in this document were assembled from the documented field tables.
- **Base URL** is not stated anywhere in the guide (Section 1.1).
- **Authorization header format** is described only as 'the Secret Key' (Section 2). Confirm whether a `Bearer` prefix is needed.
- **Account type list.** `01002` appears on some pages but was not shown on the QR Code page (Section 4.3).
- **Token-charge column** on the Response page is an icon, interpreted as Yes/No in Section 3.1.
- **Amount format.** The note 'do not include 0 and commas' for `checkAmount.amount` is ambiguous (Section 4.1).
- **Image endpoint payload.** Condition keys are listed at the top level of the `payload` field, unlike the JSON endpoints (Section 5.2.1).
- **Queue API callbacks.** Delivery, retry and authentication details are not documented (Section 6.1.3).
- **Sample names.** Names in sample data were translated to English; the original samples use Thai-language names.

Source pages: slip2go.com/guide/rest-api/{qr-code, image, base64, image-url, promptpay-qr, promptpay-qr-url, slip-history, account-info}, slip2go.com/guide/queue-api/qr-code, slip2go.com/guide/authentication and slip2go.com/guide/response.